Institut National Des Sciences Appliquées
ROUEN NORMANDIE

Rapport de projet

Projet intégratif développement orienté objet

Étudiants :
Bonneviale Thomas
El Omari Bouya Lina
Sanson Dylan
Sourdrille Nathan

Responsables de projet :
Géraldine Del Mondo
Robin Condat
Nicolas Delestre
Nicolas Malandain
Laurent Vercouter

Table des matières

1 Introduction **2**

2 Base de données **3**
2.1 Passage modèle E-A vers relationnel . . . 3
2.2 Implémentation de la persistance avec JDBC . . . 4
2.3 Enregistrement des données : EnregistreurBD.java . . . 4
2.4 Lecture des données : LecteurBD.java . . . 4
2.5 Fichier de test et démonstration . . . 4

3 Compilation **5**
3.1 Organisation . . . 5
3.2 Etude du problème . . . 5
3.3 Rédaction de la grammaire . . . 5
3.4 Implémentation en Java . . . 6
3.5 Configuration du fichier Javacc . . . 6
3.6 Vérification de l'analyseur syntaxique . . . 6
3.7 Implémentation de l'interpréteur . . . 7
3.8 implémentation du lecteur . . . 7

4 Design Pattern **8**
4.1 Objectifs du pattern . . . 8
4.2 Mise en œuvre . . . 8
4.3 Extension à un nouvel univers . . . 8

5 Conclusion **10**

Chapitre 1

Introduction

Ce projet est dans la continuité du projet ITI Aventure effectué dans l'EC Programmation Avancée en Java. Ce projet consistait à créer un jeu d'aventure au tour par tour. Il est composé d'un monde contenant des entités de structuration spatiale (Pièce, Porte, Objet...) et d'entités vivantes interagissant dans le monde (Monstre, Humain...). Les objectifs de ce projet qui regroupe différents cours étudiés durant l'année étaient les suivants : inclure une persistance en base de données des éléments du jeu, intégrer une nouvelle fonctionnalité qu'est la lecture d'un fichier de configuration en utilisant un langage fonctionnel pour décrire l'organisation du jeu ainsi que l'implémentation d'un Design Pattern permettant d'accroître le potentiel de réutilisation du jeu (illustré dans le projet par la mise en place d'une autre version du jeu, "ITISpaceOpera" qui se déroule dans l'espace).

Matrice des tâches réalisées

La consigne indiquant clairement que chaque membre du groupe devait participer à toutes les parties, nous nous sommes organisés en conséquence, comme le montre la matrice des tâches réalisées.

Tâches	**Bonneviale Thomas**	**El Omari Bouya Lina**	**Sanson Dylan**	**Sourdrille Nathan**
Base de données	X	X	X	X
Compilation	X	X	X	X
Design Pattern	X	X	X	X
Rédaction du rapport	X	X	X	X

Chapitre 2

Base de données

Dans cette partie du projet ITI Aventure, nous avons implémenté la persistance des données en utilisant une base de données relationnelle. L'objectif principal était de comprendre et d'appliquer un mapping objet-base de données, c'est-à-dire relier les classes Java du jeu (comme Monde, Piece, Porte) à des tables relationnelles.

2.1 Passage modèle E-A vers relationnel

Un modèle Entité-Association nous a été fourni.

Dans notre cas, le modèle EA définissait des entités :
— Monde, Piece, Porte, JoueurHumain, PiedDeBiche
Et des relations entre ces entités :
— Contient,
— Possède
Ces relations sont accompagnées de cardinalités qui précisent combien d'éléments peuvent être liés entre eux. Par exemple, une Porte est liée à exactement deux Pieces, tandis qu'une Piece peut contenir aucun ou plusieurs PiedDeBiche.

Une fois les entités principales identifiées, nous avons pu créer un modèle relationnel, autrement dit une version de notre modèle adaptée à une base de données relationnelle.

Chaque entité et chaque relation du modèle EA devient une table SQL :
— Monde : identifie un monde par son nom.
— Piece : associée à un monde, représente un lieu du jeu.
— Porte : relie deux pièces avec un état (ouvert, fermé, verrouillé).
— JoueurHumain : joueur présent dans le monde, avec ses points de vie et de force.
— PiedDeBiche : objet manipulable par le joueur ou présent dans une pièce.
— ContientPDB : association entre une pièce et les objets qu'elle contient.
— PossedePDB : association entre un joueur et les objets qu'il possède.
Chaque table possède une clé primaire (souvent le nom des entités), et les relations entre les tables sont assurées grâce à des clés étrangères.

Cela correspond à 3 fichiers SQL :
— tables.sql : création des tables
— sup$_{t}able.sql$: $suppressiondesdonnestest_{b}d.sql$: $jeudedonnespourtester$

Nous avons choisi ce découpage en particulier de façon à ce que chaque table puisse représenter une entité ou une relation, et qu'on puisse facilement ajouter d'autres objets ou d'autres relations si besoin. Cette étape est essentielle avant d'implémenter la persistance via JDBC, car elle conditionne la façon dont les données seront lues et écrites dans la base de données.

2.2 Implémentation de la persistance avec JDBC

Une fois le modèle relationnel défini, nous avons implémenté la persistance en Java à l'aide de JDBC. Cela a permis d'établir une connexion entre Java et une base de données relationnelle, et d'exécuter des requêtes SQL pour lire ou modifier des données.

Nous avons utilisé des objets Connection pour se connecter à la base PostgreSQL, des PreparedStatement pour exécuter les requêtes (INSERT, SELECT), et des ResultSet pour récupérer les résultats des requêtes de lecture.

2.3 Enregistrement des données : EnregistreurBD.java

La classe EnregistreurBD permet d'enregistrer un monde Java dans la base de données. Elle parcourt les entités du monde et exécute les requêtes SQL nécessaires pour remplir les tables. Chaque élément est inséré dans un ordre cohérent pour respecter les dépendances (par exemple, une pièce est insérée avant les portes qui la relient).

Les objets sont insérés grâce à des requêtes SQL paramétrées, comme :

— `INSERT INTO Piece (nomPiece, monde) VALUES (?, ?);`

Les relations entre entités sont également insérées via les tables de liaison ContientPDB et PossedePDB.

2.4 Lecture des données : LecteurBD.java

La classe LecteurBD a pour rôle de lire les données enregistrées dans la base et de recréer les objets Java du jeu.
Chaque méthode correspond à une table, par exemple :

- lecteurMonde() : lit la table Monde et crée un objet Monde par instanciation
- lecteurPiece() : lit les pièces et les associe au monde
- lecteurPorte() : lit les portes et les relie aux bonnes pièces
- lecteurJoueurHumain() : crée le joueur et récupère les objets qu'il possède

Les relations sont restaurées en retrouvant les bons objets grâce aux noms stockés dans la base, puis en les associant correctement dans le monde Java.

2.5 Fichier de test et démonstration

Un main a été mis en place pour tester l'enregistrement et la lecture d'un monde à partir de la base de données. Le fichier test$_bd.sqlpermetd'insrerdesdonnesprdfiniesdanslestablesafindevrifierquelechargementdepui$

Nous avons également pu tester le bon fonctionnement de l'enregistrement en Java en comparant les lignes insérées dans la base avec les entités du monde initial.

Cette partie du projet nous a permis de découvrir et mettre en place :

- la communication entre Java et une base relationnelle via JDBC ;
- l'utilisation de PreparedStatement, ResultSet et Connection ;
- la construction d'un lien entre des objets Java et des lignes de tables SQL ;
- l'importance de bien structurer les données dans la base de donnée pour faciliter les opérations de lecture et d'écriture.

Chapitre 3

Compilation

3.1 Organisation

La partie compilation consistait à créer une version améliorée d'un des lecteurs, le lecteur de fichier texte qui permettait de charger un monde, afin de le rendre plus générique.

Pour cela nous avons appliqué les principes et méthodes appris lors du cours de compilation suivi ce semestre. Nous avons donc décomposé ce sous-projet en différentes parties donc nous avons pu nous occuper en parallèle ou successivement selon les besoins. Le découpage qui a été utilisé est le suivant :

- Etude du problème ;
- rédaction de la grammaire et conformité pour Javacc ;
- implémentation des classes de l'AST et des classes d'exceptions en Java ;
- configuration du fichier Javacc ;
- vérification de la sortie de l'analyseur syntaxique ;
- implémentation de l'interpréteur (et analyseur sémantique) ;
- implémentation du lecteur et intégration au Main du projet.

3.2 Etude du problème

Nous avons commencé cette partie du projet par analyser et comprendre la structure du fichier que nous devions interpréter. Nous disposions d'un exemple sur Moodle ainsi que de la description de chaque mot qui pouvait exister dans ce fichier. Le fichier se présentait de la façon suivante (un extrait seulement) :

```
m1 = monde("LeMondeImpitoyableDeITI")
bureau_nicos = piece(m1, "BureauDesNicolas")
```

Une autre contrainte était donnée : un fichier ne peut créer qu'un seul monde.

L'objectif final du lecteur étant de créer un monde (et de respecter les contraintes liées à l'interface lecteur), le programme a développé était donc un interpréteur et non un compilateur.

3.3 Rédaction de la grammaire

Notre méthodologie a été itérative, nous sommes partis de l'axiome et avons succesivement créé les éléments donc nous avions besoin pour décrire chaque ligne. Notre grammaire est donc la suivante, l'axiome est ici DeclarationMultiple :

```
DeclarationMultiple -> DeclarationSimple DeclarationSimple*
DeclarationSimple -> identifiant = Fonction
Fonction -> nomFonction ( SuiteArguments )
SuiteArguments -> ArgumentSimple ArgumentSimple*
ArgumentSimple -> identifiant|chiffre|"chaineDeCaractere"|enumeration|fonction
```

La grammaire n'étant pas récursive à gauche et ne manipulant pas ϵ elle est compatible avec Javacc et nous permet donc de passer à l'étape suivante.

3.4 Implémentation en Java

A partir de la grammaire nous avons pu définir la forme que prendrait notre AST et donc définir les classes représentant les noeuds et les feuilles de cet AST :

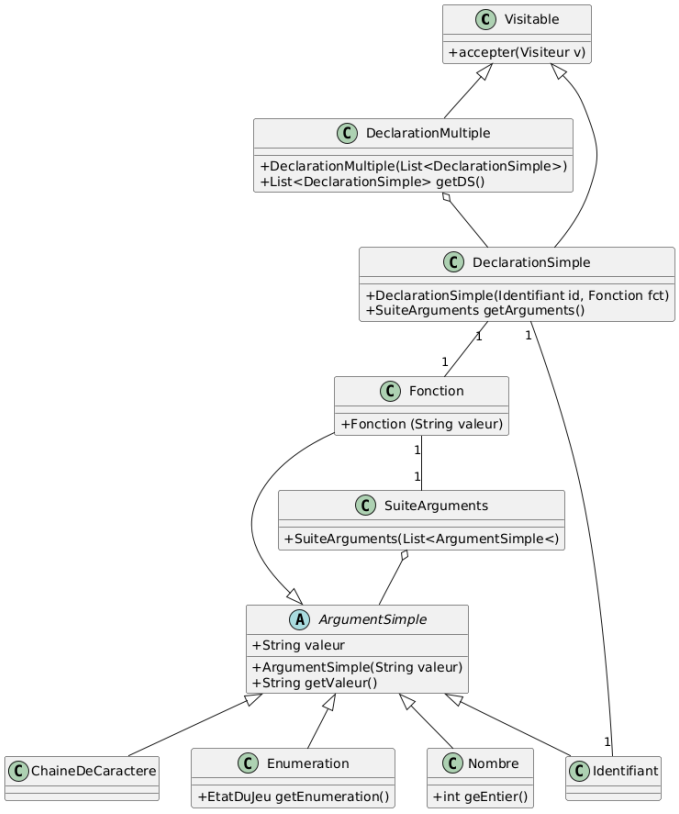

FIGURE 3.1 – Diagramme UML illustrant notre AST

A partir de là nous nous sommes séparés en deux groupes : un groupe sur l'implémentation en Java et l'autre sur la configuration du fichier Javacc.

Le groupe implémentant les classes en Java a suivi le diagramme de classe définit ci-dessus, ce qui n'a donc pas été très long. Nous avons fait le choix de créer une classe abstraite pour les ArgumentsSimple, ce qui nous a fait implémenter une classe Chiffre et ChaineDeCaractere qui sont des doublons des classes déjà existantes en Java mais qui, selon nous, permettait une meilleure lecture de l'AST.

3.5 Configuration du fichier Javacc

Afin de configurer le fichier Javacc, nous sommes partis de la base du fichier Javacc de la calculatrice présentée lors du cours de compilation. Nous l'avons complété par une SKIP qui permet d'ignorer certains caractères (espaces, sauts de lignes, tabulations, etc.) qui n'ont pas d'intérêt dans la lecture de ce fichier.

Nous avons également transformé les terminaux de la grammaire en expression rationnelle afin de définir les TOKEN qui seront utilisés par l'analyseur lexical.

La dernière partie à consisté à écrire le code Java associé à chaque règle de la grammaire en s'appuyant sur l'implémentation de l'AST définit précédemment par le diagramme de classe. La grammaire étant relativement simple, cette partie a été relativement rapide.

3.6 Vérification de l'analyseur syntaxique

Avant de commencer l'implémentation de l'interpréteur, le groupe qui a implémenté les classes en Java a testé la validité de l'AST produit par l'analyseur syntaxique à partir de quelques lignes du fichier d'exemple. Cette étape nous a permis de nous rendre compte que nous n'avions pas pris en compte les serrures, c'est-à-dire que nous n'avions pas compté qu'une fonction pouvait faire office d'argument

(ArgumentSimple) dans une autre fonction. Cela a conduit à une modification de la grammaire, du diagramme de classe et du fichier Javacc. Cette modification n'a heureusement pas induit de changement majeur dans le code.

Une fois l'AST conforme au fichier de description nous avons pu commencer à implémenter l'interpréteur.

3.7 Implémentation de l'interpréteur

Pour l'interpréteur nous avons fait le choix de le fusionner avec l'analyseur sémantique. La taille du projet permettait une fusion sans que l'implémentation ne soit trop lourde (et sur conseil de M. Delestre).

Une grande partie du temps que nous avons consacré l'interpréteur a consisté à analyser le code des interpréteurs de la Calculatrice et du LightGrep.

Certaines classes qui étaient prévues pour être Visitable n'ont finalement pas été utilisée avec le Design Pattern Visiteur/Visitable, l'accumulation de problème dans le développement de l'interpréteur ne nous a pas permis de suivre correctement l'utilisation de ce Design Pattern.

Les tests de l'interpréteur se sont fait au fur et à mesure de la prise en charge des différentes fonctions du fichier descripteur par l'interpréteur. Cette méthodologie nous a fait gagner un temps précieux.

3.8 implémentation du lecteur

L'interface lecteur nous contraignait simplement à pouvoir récupérer les conditions de fin ainsi que le Monde qui a été chargé. Ce qui nous a poussé à intégrer ces fonctions directement dans l'interpréteur. Ces deux fonctions de l'interpréteur nous ont fait gagner du temps lors du développement du lecteur.

L'implémentation du lecteur s'est effectuée rapidement et sans problèmes majeurs, ce qui nous a permis d'intégrer le lecteur assez rapidement dans le Main du projet et d'avoir un nouveau lecteur fonctionnel.

Chapitre 4

Design Pattern

Dans le cadre de la refonte architecturale du projet ITIAventure, nous avons introduit le design pattern **Factory**. Ce patron de création a été choisi pour séparer clairement la logique d'instanciation des entités (comme les pièces, portes ou créatures) du reste de l'application. En centralisant cette logique dans une classe dédiée, le code devient plus modulaire, maintenable et extensible, facilitant notamment l'ajout de nouveaux univers de jeu.

4.1 Objectifs du pattern

Le pattern *Factory* a pour vocation de proposer une interface standardisée pour la création d'objets, tout en dissimulant les classes concrètes qui les implémentent. Ce découplage permet à l'application de rester indépendante des détails de construction des objets manipulés.

Dans notre cas, ce besoin s'est exprimé lors de la lecture du fichier de description du labyrinthe. Initialement, la classe `LecteurDescription` invoquait directement les constructeurs des entités, ce qui liait fortement cette classe à une version particulière de l'univers du jeu.

4.2 Mise en œuvre

Nous avons conçu une classe `ITIAventureFactory` regroupant les méthodes de création des entités suivantes :

- `creationMonde()` → `Monde`
- `creationPiece()` → `Piece`
- `creationPorte()` → `Porte`
- `creationMonstre()` → `Monstre`
- `creationCle()` → `Cle`
- `creationSerrure()` → `Serrure`

La classe `LecteurDescription` a été modifiée afin de déléguer la création de ces entités à la factory, plutôt que de les instancier directement. Ce changement rend possible la sélection dynamique de la factory à utiliser, en se basant sur le contenu du fichier de description (notamment sa première ligne).

4.3 Extension à un nouvel univers

Dans le but de proposer une version alternative de l'univers du jeu, nous avons introduit de nouvelles entités propres à un contexte de science-fiction. Ces classes ont été développées dans le package `fr.insarouen.iti.prog.aventure.spaceopera`, chacune héritant de leur équivalent dans l'univers d'origine :

- `Galaxie` (héritant de `Monde`)
- `VaisseauSpatial` (héritant de `Piece`)
- `Teleporteur` (héritant de `Porte`)
- `Alien` (héritant de `Monstre`)
- `Badge` (héritant de `Cle`)

— `LecteurBadge` (héritant de `Serrure`)

Afin d'intégrer ces entités dans notre moteur de jeu tout en conservant la logique de création centralisée, nous avons implémenté une classe `ITISpaceOperaFactory`, conforme à l'interface définie par la factory abstraite. Cette classe redéfinit les méthodes de création telles que :

— `creationMonde()` → `Galaxie`
— `creationPiece()` → `VaisseauSpatial`
— `creationPorte()` → `Teleporteur`
— `creationMonstre()` → `Alien`
— `creationCle()` → `Badge`
— `creationSerrure()` → `LecteurBadge`

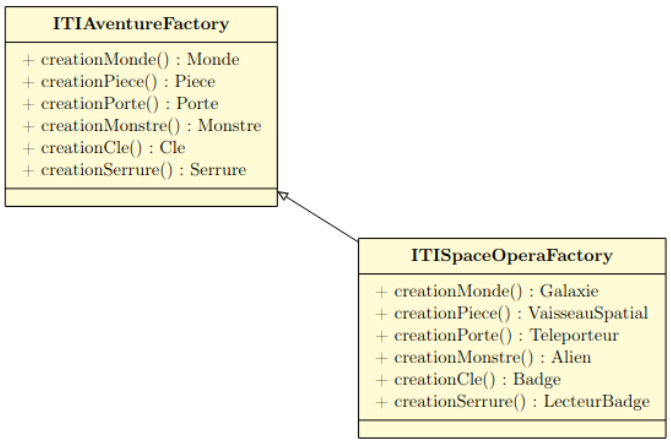

FIGURE 4.1 – Diagramme UML illustrant l'héritage entre `ITIAventureFactory` et `ITISpaceOperaFactory`

Ce découplage rend le moteur de jeu entièrement générique : il suffit de changer la factory utilisée pour adapter l'expérience à un nouvel univers.

Chapitre 5

Conclusion

Le projet ITI Aventure nous a permis de mettre en pratique plusieurs compétences acquises durant l'année, en particulier la persistance des données avec JDBC, la création d'un interpréteur via JavaCC, et l'utilisation de design patterns pour améliorer la structure du code.

Nous avons conçu une base relationnelle adaptée au jeu, développé un système de lecture et d'écriture des données en Java, implémenté un langage de description fonctionnel pour créer un monde, et généralisé notre code pour permettre des extensions comme ITISpaceOpera.

Au-delà des aspects techniques, ce projet à développer notre sens de la communication en équipe, à résoudre des problèmes concrets de développement logiciel, et à concevoir une architecture claire et évolutive. Il a constitué une expérience plutôt riche, d'une part par le fait qu'il s'agit du projet informatique le plus concret de l'année et d'autre part car le temps très limité à notre disposition nous contraignait à être prêt pour la date de rendu malgré notamment les contraintes de révision des examens.